



PHP operators 30m

PHP operators are used to perform operations on variables and values. We will discuss arithmetic operators, logical operators and relational and conditional operators in this section.

PHP arithmetic operators

PHP operators perform typical mathematical calculations as they do in other programming languages.

Operator	Name	Description	Example	Result
+	Addition	Adds values together	100 + 50	150
-	Subtraction	Subtracts the right operand from the left	100 - 50 50 - 100	50 -50
*	Multiplication	Multiplies the values	5 * 10	50
/	Division	Divides the left operand by the right operand	10 / 2	5

[Skip to main content](#)

%	Modulus	Divides the left operand by the right operand and returns the remainder	10 % 4	2
			10 % 5	0
			10 % 3	1
++	Increment	Increases the value of the operand by 1	\$num = 10 \$num++	11
--	Decrement	Decreases the value of the operand by 1	\$num = 10 \$num--	9
**	Exponentiation	Raise a value to the power of	\$num = 10 \$num**2	100

PHP expressions

Most arithmetic expressions in PHP comprise two sides – the left-hand side (result) and the right-hand side (the expression), coupled together with one of the arithmetic operators. For example:

```
// [left-hand side] [assignment operator] [right-hand side];
```

```
$a = 1 + 2; // simple calculation
```

Some expressions, like those that use unary operators, only have a left-hand side. For example:

```
// set some values
```

[Skip to main content](#)

```
$b = 10;
```

```
$a = 0;
```

`$b = $b + $a++;` // ++ after the variable increments it after the + expression, so it is still 0 when used.

`$b = $b + ++$a;` //++ before the variable increments it before the + expression, so it becomes 2 before the assignment to \$b.

PHP logical operators

PHP logical operators are used to combine conditional statements. A conditional statement will be based on a condition which can be met or not, so the result of a conditional statement will be true or false. This means a PHP logical operator will return a Boolean (true or false).

Operator	Name	Syntax	Description	Example
and	And	\$x and \$y	True if both \$x and \$y are true	<pre>\$x = 20; \$y = 10; if (\$x == 20 and \$y == 10) Result: TRUE if (\$x == 20 and \$y == 30) Result: FALSE</pre>

[Skip to main content](#)

or	Or	\$x or \$y	True if either \$x or \$y is true	\$x = 20; \$y = 10; if (\$x == 20 or \$y == 50) Result: TRUE if (\$x == 10 or \$y == 20) Result: FALSE
xor	Xor	\$x xor \$y	True if either \$x or \$y is true, but not both	\$x = 20; \$y = 10; if (\$x == 20 xor \$y == 50) Result: TRUE if (\$x == 20 xor \$y == 10) Result: FALSE

[Skip to main content](#)

&&	And	\$x && \$y	True if both \$x and \$y are true	\$x = 20; \$y = 10; if (\$x == 20 && \$y == 10) Result: TRUE if (\$x == 20 && \$y == 30) Result: FALSE
	Or	\$x \$y	True if either \$x or \$y is true	\$x = 20; \$y = 10; if (\$x == 20 \$y == 50) Result: TRUE if (\$x == 10 \$y == 20) Result: FALSE

[Skip to main content](#)

!	Not	!\$x	True if \$x is not true	\$x = 20; if (\$x !== 50) Result: TRUE if (\$x !== 20) Result: FALSE
---	-----	------	-------------------------	--

The reason there are two **and** operators and two **or** operators is because of precedence. Operator precedence decides which operator is performed first in an expression that has more than one operator.

The **and** operator has higher precedence compared to the **&&** operator, so the **and** operator will be performed earlier than the **&&** operator in an expression.

The **or** operator has higher precedence compared to the **||** operator, so the **or** operator will be performed earlier than the **||** operator in an expression.

Operator precedence is important to be aware of because you need to know which operator will be performed first.

For example, the expression `2 + 8 * 2` will equal 18 because the `*` operator has higher precedence than the `+` operator.

You can learn more about operator precedence in the PHP manual.

PHP comparison operators

PHP comparison operators are used to compare two values, which can be either numbers or strings. Once an expression has been compared using the comparison operators, the output will show the result in Boolean form – that is, true or false.

Operator	Description	True example	False example
Skip to main content			

<	Less than	3 < 8	8 < 3
>	Greater than	4 > 2	2 > 4
==	Equal to	7 == 7	3 == 9
===	Equal in value and type	7 === 7	"7" === 7
!=	Not equal or not of the same type	7 != "7"	7 != 7
<=	Less than or equal to	5 <= 5	8 <= 6
>=	Greater than or equal to	7 >= 3	1 >= 2
!=	Not equal to	5 != 6	3 != 3



☒ Operators 1

Non-Assessed

What is the correct comparison operator to use to check if \$a is equal to \$b?

- ☐ &&
- ☐ !!
- ☐ ||
- ☐ ==

Submit

[Skip to main content](#)



☒ Operators 2

Non-Assessed

What is the correct comparison operator to use to check if \$a is NOT equal to \$b?
var_dump (\$a _____ \$b)

- ☐ &&
- ☐ !=
- ☐ ||
- ☐ ==

Submit



☒ Operators 3

Non-Assessed

[Skip to main content](#)

Complete the code below so the output reads 'Welcome to Open Polytechnic' if \$x is greater than \$y.

```
<?php
$x = 100;
$y = 50;

if (_____)
{
    echo "Welcome to Open Polytechnic";
}
?>
```

- ☐ \$x > \$y
- ☐ \$x = \$y
- ☐ \$x == \$y
- ☐ \$x < \$y

Submit



☒ Operators 4

Non-Assessed

[Skip to main content](#)

What will be the output for the code below?

```
<?php $x = 5; $y = 2; echo $x ** $y; ?>
```

- ☐ 10
- ☐ 25
- ☐ 7
- ☐ 30

Submit



☒ Operators 5

Non-Assessed

What will be the output for the code below?

```
<?php $x = 12; $y = 5; echo $x % $y; ?>
```

- ☐ 2
- ☐ 7
- ☐ 0
- ☐ Expression cannot be performed

Skip to main content



☒ Operators 6

Non-Assessed

What logical operator would you use to check whether either \$x or \$y are true, but not both?

- ☐ xor
- ☐ or
- ☐ ||
- ☐ !

Submit



Thoughts on this page?

